LAPORAN PRAKTIKUM MOBILE PROGRAMMING

Laporan ini disusun untuk memenuhi tugas
Praktikum Mobile Programming Modul 10

Dosen Pengampu:

Khadijah Fahmi Hayati Holle, S.Kom., M.Kom



Disusun oleh:

Muhammad Rizqullah Saputra
240605110102

PRODI TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISLAM NEGERI
MAULANA MALIK IBRAHIM MALANG
2026

PENDAHULUAN

Modul 10 membahas penerapan desain antarmuka, navigasi antarhalaman, dan integrasi API dengan JSON serialization pada Flutter. Laporan ini menguraikan tahapan pembuatan aplikasi mulai dari pengaturan proyek, penyusunan halaman Home, pengambilan data game, hingga pembuatan halaman Detail dan konfigurasi named route. Urutan pembahasan mengikuti materi praktikum agar hubungan antara antarmuka, model data, dan proses komunikasi dengan API dapat dipahami secara bertahap.

Aplikasi yang dibahas adalah game_app, yaitu katalog game yang memperoleh informasi dari layanan FreeToGame. Halaman Home digunakan untuk menyajikan daftar permainan, sedangkan halaman Detail memuat informasi dari game yang dipilih, berupa gambar utama, kebutuhan sistem minimum, screenshot, dan deskripsi. Kode di dalam direktori lib dibagi menjadi folder model, view, dan viewmodel untuk memisahkan representasi data, tampilan aplikasi, serta fungsi pengambilan data.

Tujuan praktikum ini adalah memahami penyusunan GUI Flutter yang rapi dan responsif, menerapkan navigasi dengan named route, serta mempelajari konversi data JSON menjadi objek Dart dan sebaliknya. Selain itu, praktikum memperkenalkan penggunaan paket http untuk mengakses API eksternal dan penanganan proses asinkron melalui FutureBuilder. Kebutuhan pelaksanaan mencakup Flutter SDK, Android Studio atau VS Code, emulator atau perangkat Android, dan koneksi internet.

PERSIAPAN PROYEK DAN DEPENDENSI

A. Membuat Proyek dan Struktur Folder

Pengerjaan dimulai dengan membuat proyek Flutter bernama game_app. Pembagian folder dilakukan sejak awal supaya setiap bagian kode memiliki fungsi yang jelas. Folder model menyimpan kelas data, view menampung widget halaman, dan viewmodel berisi fungsi komunikasi dengan API.

1. Buat proyek game_app melalui Android Studio atau VS Code.
2. Tambahkan folder model, view, dan viewmodel di dalam direktori lib.
3. Tempatkan berkas pada folder yang sesuai dengan tanggung jawabnya. Pada tahap kedua, halaman Detail ditambahkan ke proyek yang sama.

```
lib/
  main.dart
  model/
    game.dart
    detailgame.dart
    detailgame.g.dart
  view/
    home.dart
    detail.dart
  viewmodel/
    fetchgame.dart
```

Struktur di atas menunjukkan susunan akhir setelah kedua tahap selesai disusun. Berkas detailgame.g.dart bukan ditulis secara manual, melainkan dihasilkan ketika code generator dijalankan.

B. Menambahkan Dependensi pada pubspec.yaml

Dependensi tambahan mendukung pemanggilan API dan pengolahan data. Paket http digunakan untuk permintaan jaringan, json_annotation menyediakan anotasi model, sedangkan json_serializable dan build_runner menghasilkan kode konversi JSON. Paket readmore dipakai untuk menampilkan deskripsi panjang dalam bentuk yang dapat diperluas.

1. Tambahkan http, json_annotation, dan readmore pada bagian dependencies.
2. Masukkan build_runner dan json_serializable pada bagian dev_dependencies.
3. Simpan perubahan lalu jalankan flutter pub get dari direktori proyek. Versi berikut mengikuti contoh yang tercantum pada modul.

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.1.0
  json_annotation: ^4.9.0
  readmore: ^2.2.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  build_runner: ^2.4.8
  json_serializable: ^6.7.1
```

C. Konfigurasi Awal main.dart

Berkas main.dart berfungsi sebagai titik masuk aplikasi. Fungsi main() menjalankan MyApp, sedangkan MyApp mengembalikan MaterialApp. Pada tahap awal, properti home langsung diarahkan ke Home(); konfigurasi ini kemudian diganti dengan daftar route ketika halaman Detail sudah tersedia.

1. Impor flutter/material.dart, flutter/services.dart, dan view/home.dart.
2. Tuliskan fungsi main() yang memanggil runApp(const MyApp()). Buat MyApp sebagai turunan StatelessWidget.
3. Atur status bar agar transparan dengan ikon berwarna gelap melalui SystemChrome.setSystemUIOverlayStyle().
4. Gunakan title: 'Flutter Game Store', debugShowCheckedModeBanner: false, dan home: Home() pada MaterialApp awal.

HALAMAN HOME

A. Membuat Widget Home dan State Awal

Home menggunakan StatefulWidget beserta kelas _HomeState. Data daftar game dipersiapkan dengan tipe Future<List<Game>> karena permintaan jaringan tidak selesai secara langsung. Method initState() memanggil loadGames() ketika state pertama kali dibuat.

1. Buat berkas view/home.dart dan kelas Home beserta _HomeState.
2. Deklarasikan late Future<List<Game>> gameList pada _HomeState.
3. Panggil super.initState() lalu loadGames() di dalam initState().
4. Isi loadGames() dengan pemanggilan fetchGames() dan kembalikan Future yang disimpan.

```
late Future<List<Game>> gameList;

@override
void initState() {
  super.initState();
  loadGames();
}

Future<List<Game>> loadGames() async {
  gameList = fetchGames();
  return gameList;
}
```

Kata kunci late menunjukkan bahwa variabel akan diisi sebelum digunakan. Tampilan Home disusun menggunakan Scaffold dengan latar Colors.amberAccent.shade400, SafeArea, dan Column. Padding horizontal 10 serta vertikal 5 memberi jarak antara isi halaman dan tepi layar.

B. Membuat Kolom Pencarian (Search Bar)

Kolom pencarian disusun melalui fungsi _searchBar() yang mengembalikan TextField. Komponen ini menjadi elemen pertama pada Column, diikuti SizedBox setinggi 10 sebagai pemisah dari daftar game. Contoh modul pada bagian ini mengatur tampilan input, tetapi belum menambahkan logika untuk menyaring data berdasarkan kata yang diketik.

1. Letakkan fungsi _searchBar() di luar _HomeState.
2. Gunakan fillColor: Colors.blue.shade50 dan filled: true untuk memberi latar pada kolom.
3. Atur OutlineInputBorder dengan BorderRadius.circular(20) dan BorderSide.none.
4. Tambahkan ikon Icons.search_outlined, hintText: 'Cari game', serta padding agar isi kolom tidak menempel pada tepi.

C. Membuat Tampilan Setiap Item Game

Setiap data game ditampilkan melalui fungsi _listItem(urlCover, judul, genre). Widget Card menjadi bingkai item, sedangkan ListTile membagi isi menjadi thumbnail, judul, genre, dan ikon tambahan. Susunan ini membuat tampilan setiap entri konsisten meskipun datanya berbeda.

1. Buat fungsi _listItem() di luar kelas _HomeState.
2. Atur Card dengan Clip.antiAlias, elevation 3, dan sudut melengkung 8.
3. Gunakan SizedBox berukuran 75 x 75 pada leading untuk menempatkan Image.network di dalam ClipRRect.
4. Isi title dan subtitle dengan judul serta genre, lalu gunakan Icons.more_vert pada trailing. Atur titleAlignment menjadi ListTileTitleAlignment.center.

```
Card _listItem(String urlCover, String judul, String genre) {
  return Card(
    clipBehavior: Clip.antiAlias,
    elevation: 3,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(8)),
    child: ListTile(
      tileColor: Colors.blue.shade50,
      leading: SizedBox(
        width: 75,
        height: 75,
        child: ClipRRect(
          borderRadius: BorderRadius.circular(8),
          child: Image.network(
            urlCover, scale: 3, fit: BoxFit.cover),
        ),
      ),
      title: Text(judul),
      subtitle: Text(genre),
      trailing: const Icon(Icons.more_vert),
      isThreeLine: false,
      titleAlignment: ListTileTitleAlignment.center,
    ),
  );
}
```

D. Menampilkan Daftar Game dengan FutureBuilder

FutureBuilder<List<Game>> menghubungkan hasil permintaan data dengan tampilan halaman. Selama permintaan masih berlangsung, antarmuka menampilkan CircularProgressIndicator. Apabila terjadi kesalahan atau tidak ada data, pesan yang sesuai ditampilkan. Daftar baru dibangun ketika respons berhasil diproses.

1. Tambahkan FutureBuilder setelah kolom pencarian dan SizedBox. Contoh modul menggunakan future: fetchGames().
2. Periksa ConnectionState.waiting untuk menentukan kapan indikator loading ditampilkan.
3. Tangani snapshot.hasError dan kondisi !snapshot.hasData atau snapshot.data!.isEmpty.
4. Ambil 25 data pertama menggunakan take(25).toList(), lalu bangun daftar melalui ListView.builder yang dibungkus Expanded.
5. Tambahkan GestureDetector pada setiap kartu agar ID game dapat dikirim ke route /detail. Route tersebut didaftarkan pada tahap kedua.

```
FutureBuilder<List<Game>>(
  future: fetchGames(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Expanded(
        child: Center(child: CircularProgressIndicator()));
    } else if (snapshot.hasError) {
      return Text('Error: ${snapshot.error}');
    } else if (!snapshot.hasData || snapshot.data!.isEmpty) {
      return const Text('Tidak ada data game.');
    } else {
      final games = snapshot.data!.take(25).toList();
      return Expanded(
        child: ListView.builder(
          itemCount: games.length,
          itemBuilder: (context, index) {
            final game = games[index];
            return GestureDetector(
              onTap: () {
                Navigator.pushNamed(context, '/detail',
                  arguments: game.id);
              },
              child: _listItem(
                game.thumbnail, game.title, game.genre),
            );
          },
        ),
      );
    }
  },
)
```

SUMBER DATA DAN MODEL GAME

A. Fungsi fetchGames() pada Folder viewmodel

Fungsi fetchGames() ditempatkan di viewmodel/fetchgame.dart supaya logika pengambilan data terpisah dari widget. Endpoint /api/games mengembalikan kumpulan game dalam format JSON. Data tersebut dikonversi menjadi daftar objek Game sebelum digunakan oleh halaman Home.

1. Impor dart:convert, paket http dengan alias http, dan model/game.dart.
2. Buat fungsi asinkron dengan tipe kembalian Future<List<Game>>.
3. Kirim permintaan GET ke https://www.freetogame.com/api/games.
4. Jika status respons 200, gunakan json.decode() lalu petakan setiap elemen dengan Game.fromJson(item).
5. Lempar exception apabila respons tidak menunjukkan keberhasilan.

```
Future<List<Game>> fetchGames() async {
  final response = await http.get(
    Uri.parse('https://www.freetogame.com/api/games'));
  if (response.statusCode == 200) {
    final List<dynamic> jsonData = json.decode(response.body);
    return jsonData.map((item) => Game.fromJson(item)).toList();
  } else {
    throw Exception('Gagal mengambil data game');
  }
}
```

Penggunaan await memungkinkan fungsi menunggu respons jaringan sebelum melanjutkan pemrosesan. json.decode() mengubah teks JSON menjadi struktur Dart, sementara map() menerapkan pembentukan objek Game pada seluruh elemen daftar.

B. Kelas Game pada Folder model

Kelas Game menjadi representasi data dari endpoint daftar game. Atribut menggunakan final sehingga nilainya tidak dapat diganti setelah objek terbentuk. Factory Game.fromJson() menjalankan deserialization secara manual, termasuk pemetaan nama kunci JSON yang memakai garis bawah ke nama properti Dart.

1. Buat berkas model/game.dart.
2. Deklarasikan atribut id, title, thumbnail, shortDescription, gameUrl, genre, platform, publisher, developer, releaseDate, dan freetogameProfileUrl.
3. Gunakan int untuk id dan String untuk atribut lainnya. Sediakan konstruktor dengan parameter required.
4. Buat factory Game.fromJson(Map<String, dynamic> json) dan isi properti dari kunci yang sesuai.

```
factory Game.fromJson(Map<String, dynamic> json) {
  return Game(
    id: json['id'],
    title: json['title'],
    thumbnail: json['thumbnail'],
    shortDescription: json['short_description'],
    gameUrl: json['game_url'],
    genre: json['genre'],
    platform: json['platform'],
    publisher: json['publisher'],
    developer: json['developer'],
    releaseDate: json['release_date'],
    freetogameProfileUrl: json['freetogame_profile_url'],
  );
}
```

C. Menjalankan Program pada Halaman Home

Pemeriksaan tahap pertama berfokus pada hubungan antara fungsi pengambilan data, model Game, dan daftar kartu. Screenshot Home pada bagian hasil memperlihatkan kolom pencarian serta kartu yang berisi thumbnail, judul, dan genre. Langkah berikut digunakan sebagai acuan pemeriksaan tampilan halaman tersebut.

1. Jalankan aplikasi pada emulator atau perangkat Android melalui flutter run.
2. Amati indikator loading ketika data sedang diminta dari API.
3. Periksa kemunculan thumbnail, judul, dan genre pada kartu game.
4. Pastikan daftar dapat digulir dan pesan kesalahan diperhatikan apabila koneksi atau permintaan API gagal.

HALAMAN DETAIL

A. Widget Detail dan Method fetchData()

Tahap kedua melanjutkan proyek yang sama dengan menambahkan view/detail.dart. Sesuai modul, Detail dibuat sebagai StatelessWidget dengan parameter wajib gameTerpilih bertipe int. Nilai ini adalah ID yang dikirim dari Home, bukan seluruh objek game. Halaman Detail kemudian meminta informasi lengkap berdasarkan ID tersebut.

1. Buat Detail sebagai turunan StatelessWidget dan deklarasikan final int gameTerpilih.
2. Tambahkan konstruktor const Detail({super.key, required this.gameTerpilih}).
3. Impor model/detailgame.dart, viewmodel/fetchgame.dart, dan paket readmore.
4. Tulis fetchData() untuk memanggil fetchDataFromAPI(gameTerpilih), lalu konversikan hasilnya menjadi DetailGame.

```
Future<DetailGame> fetchData() async {
  final jsonData = await fetchDataFromAPI(gameTerpilih);
  return DetailGame.fromJson(jsonData);
}
```

B. Model DetailGame dengan JSON Serialization

Berbeda dari model Game yang mengonversi JSON secara manual, model detail memanfaatkan anotasi @JsonSerializable(). Tiga kelas digunakan, yaitu MinimumSystemRequirements, Screenshot, dan DetailGame. Method fromJson() serta toJson() pada masing-masing kelas terhubung ke fungsi yang dihasilkan otomatis oleh build_runner.

1. Buat model/detailgame.dart dan impor package:json_annotation/json_annotation.dart.
2. Tambahkan deklarasi part 'detailgame.g.dart'; agar berkas hasil generator terhubung dengan model.
3. Susun MinimumSystemRequirements dengan atribut os, processor, memory, graphics, dan storage; susun Screenshot dengan id dan image.
4. Pada DetailGame, gunakan atribut yang mengikuti kunci API sebagaimana contoh modul, termasuk short_description, game_url, release_date, freetogame_profile_url, dan minimum_system_requirements.
5. Sediakan konstruktor serta factory fromJson() dan method toJson() pada ketiga kelas.
6. Jalankan flutter pub run build_runner build dari direktori proyek untuk menghasilkan detailgame.g.dart.

Cuplikan berikut menunjukkan pola serialisasi pada kelas Screenshot. Pola pemanggilan fungsi generator yang sama diterapkan pada MinimumSystemRequirements dan DetailGame, dengan daftar atribut sesuai data masing-masing.

```
import 'package:json_annotation/json_annotation.dart';

part 'detailgame.g.dart';

@JsonSerializable()
class Screenshot {
  final int id;
  final String image;

  Screenshot({required this.id, required this.image});

  factory Screenshot.fromJson(Map<String, dynamic> json) =>
      _$ScreenshotFromJson(json);
  Map<String, dynamic> toJson() => _$ScreenshotToJson(this);
}
```

Dalam model DetailGame, minimum_system_requirements bertipe MinimumSystemRequirements dan screenshots bertipe List<Screenshot>. fromJson() membentuk objek Dart dari Map, sedangkan toJson() menghasilkan representasi Map yang dapat digunakan dalam proses serialisasi. Pada contoh modul, nama properti detail dipertahankan sama dengan kunci API sehingga pemetaan tambahan dengan @JsonKey tidak diperlukan untuk nama tersebut.

C. Fungsi fetchDataFromAPI(int idGame)

Fungsi ini ditambahkan pada viewmodel/fetchgame.dart untuk mengambil satu data game. Berbeda dari endpoint daftar, endpoint detail menggunakan parameter id dan mengembalikan objek JSON. Tipe kembalian fungsi adalah Future<Map<String, dynamic>> agar hasilnya dapat diteruskan ke DetailGame.fromJson().

1. Tambahkan fungsi fetchDataFromAPI(int idGame) pada berkas pengambilan data.
2. Kirim HTTP GET ke https://www.freetogame.com/api/game?id=$idGame.
3. Saat respons 200, uraikan response.body dan periksa bahwa hasilnya berupa Map<String, dynamic>.
4. Kembalikan Map yang valid atau lempar exception bila format dan status respons tidak sesuai.

```
Future<Map<String, dynamic>> fetchDataFromAPI(int idGame) async {
  final response = await http.get(
    Uri.parse('https://www.freetogame.com/api/game?id=$idGame'));
  if (response.statusCode == 200) {
    final jsonData = json.decode(response.body);
    if (jsonData != null && jsonData is Map<String, dynamic>) {
      return jsonData;
    } else {
      throw Exception(
        'Data dari API tidak sesuai dengan yang diharapkan');
    }
  } else {
    throw Exception('Gagal mengambil data dari API');
  }
}
```

D. Struktur Dasar Halaman Detail

Body pada Scaffold menggunakan FutureBuilder untuk menunggu hasil fetchData(). Kondisi loading, error, dan data tidak tersedia ditangani sebelum antarmuka detail dibangun. Ketika data diterima, snapshot.data! disimpan pada variabel game untuk mengisi widget berikutnya.

1. Gunakan Scaffold dengan latar Colors.amberAccent.shade400.
2. Tambahkan FutureBuilder dengan future: fetchData() pada body.
3. Tampilkan CircularProgressIndicator ketika waiting, pesan kesalahan ketika snapshot.hasError, dan pesan data tidak tersedia jika snapshot tidak memiliki data.
4. Susun Column yang memuat header serta bagian informasi. Bagian informasi menggunakan Expanded dan SingleChildScrollView agar konten panjang dapat digulir tanpa mengubah tinggi header.

E. Header Gambar dan Tombol Kembali

Bagian teratas menampilkan thumbnail game dan tombol kembali. Stack memungkinkan tombol ditumpuk di atas gambar, sedangkan Positioned mengatur letaknya. ClipRRect membentuk sudut bawah gambar supaya tampilan header menyatu dengan bagian informasi.

1. Buat SizedBox dengan width: double.infinity yang membungkus Stack.
2. Tampilkan game.thumbnail menggunakan Image.network dengan height 300 dan fit: BoxFit.cover.
3. Atur radius bawah kiri dan kanan menjadi 25 melalui ClipRRect.
4. Tambahkan tombol lingkaran pada top 25 dan left 10 dengan ikon Icons.arrow_back_ios_outlined.
5. Gunakan GestureDetector dan Navigator.pop(context) untuk kembali ke Home.

F. Screenshot dan Spesifikasi Minimum

Informasi kebutuhan sistem disusun dalam panel berlatar Colors.blue.shade50 dengan sudut membulat 10. Setiap baris menampilkan nama komponen, tanda titik dua, dan nilainya. Screenshot dari API ditampilkan setelah kebutuhan sistem melalui daftar horizontal; gambar ini merupakan konten game, berbeda dari screenshot aplikasi yang dilampirkan sebagai bukti praktikum.

1. Buat _requirementRow(String title, String value) menggunakan Row dengan crossAxisAlignment: CrossAxisAlignment.start.
2. Gunakan tiga Expanded dengan perbandingan flex 3 : 1 : 8 untuk label, tanda titik dua, dan nilai.
3. Isi lima baris kebutuhan sistem: OS, Processor, Memory, Graphics, dan Storage dari game.minimum_system_requirements.
4. Buat _itemList(String url) berupa SizedBox berlebar 250, ClipRRect dengan radius 15, dan Image.network.
5. Tambahkan ListView dengan scrollDirection: Axis.horizontal dalam Container setinggi 200. Contoh modul memanggil tiga screenshot melalui indeks 0, 1, dan 2, dengan jarak 15 antar gambar.

Pemanggilan indeks tersebut mengasumsikan bahwa data game memiliki sedikitnya tiga screenshot. Ketersediaan gambar dan spesifikasi bergantung pada respons API. Karena itu,

game yang dipakai untuk dokumentasi hasil perlu dipilih berdasarkan data yang benar-benar tersedia.

G. Deskripsi dengan Paket readmore

Deskripsi yang panjang ditampilkan menggunakan ReadMoreText. Pada kondisi awal, teks dibatasi menjadi dua baris. Pengguna dapat membuka rincian dengan label more dan menutupnya kembali melalui label less. Mekanisme ini menjaga bagian informasi tetap ringkas tanpa menghilangkan isi deskripsi.

1. Impor package:readmore/readmore.dart pada detail.dart.
2. Letakkan ReadMoreText(game.description) setelah daftar screenshot.
3. Atur trimLines: 2, trimMode: TrimMode.Line, trimCollapsedText: 'more', dan trimExpandedText: 'less'.
4. Gunakan warna teks Colors.black54, jarak antarbaris 1.5, serta padding pada Container pembungkus.

NAVIGASI NAMED ROUTE

A. Mendaftarkan Route pada MaterialApp

Named route memberi nama pada setiap halaman dan memusatkan pengaturannya di MaterialApp. Setelah halaman Detail dibuat, properti home tidak lagi dipakai bersamaan dengan rute awal. Halaman Home ditempatkan pada route /, sedangkan Detail ditempatkan pada /detail.

1. Impor view/detail.dart pada main.dart.
2. Ganti home: Home() menjadi initialRoute: '/'.
3. Daftarkan route / dan /detail dalam properti routes.
4. Di dalam pembangun route /detail, ambil settings.arguments melalui ModalRoute dan cast menjadi int.
5. Teruskan ID yang diterima ke konstruktor Detail(gameTerpilih: gameId).

```
return MaterialApp(
  debugShowCheckedModeBanner: false,
  title: 'Flutter Game Store',
  initialRoute: '/',
  routes: {
    '/': (context) => const Home(),
    '/detail': (context) {
      final gameId =
          ModalRoute.of(context)!.settings.arguments as int;
      return Detail(gameTerpilih: gameId);
    },
  },
);
```

B. Mengirim dan Menerima Argumen

Argumen navigasi menggunakan game.id sebagai identitas permainan yang dipilih. Home cukup mengirim ID, sementara Detail bertanggung jawab mengambil data lengkap dari API. Pembagian ini menjaga keterkaitan kedua halaman tanpa mengharuskan seluruh data detail tersedia pada daftar Home.

1. Bungkus kartu game dengan GestureDetector pada ListView.builder.
2. Panggil Navigator.pushNamed(context, '/detail', arguments: game.id) pada onTap.
3. Tangkap argumen pada konfigurasi route /detail, lalu kirimkan ke parameter gameTerpilih.
4. Gunakan Navigator.pop(context) pada tombol kembali sehingga halaman Detail dikeluarkan dari tumpukan navigasi.

C. Menjalankan dan Menguji Aplikasi

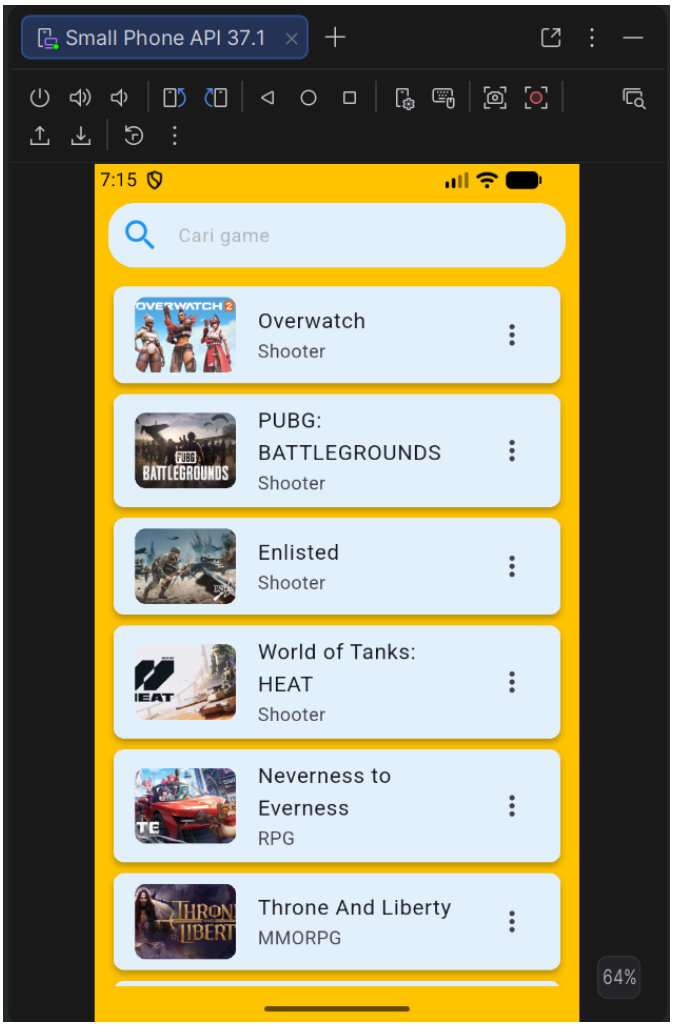
Pemeriksaan akhir mencakup perpindahan Home menuju Detail dan kesesuaian informasi game. Dua screenshot pada bagian hasil mendokumentasikan tampilan kedua halaman. Langkah berikut menjadi acuan pemeriksaan navigasi, tombol kembali, pengguliran, dan perluasan deskripsi saat aplikasi dijalankan.

1. Jalankan aplikasi dengan flutter run pada perangkat atau emulator yang digunakan.
2. Periksa daftar game pada Home setelah proses pemuatan data selesai.
3. Pilih salah satu kartu dan amati perpindahan ke route /detail.
4. Pastikan header, kebutuhan sistem, screenshot game, dan deskripsi sesuai dengan ID yang dikirim.
5. Coba perluasan deskripsi dan gulir daftar screenshot, kemudian tekan tombol kembali.
6. Ambil screenshot Home serta Detail sebagai bukti hasil praktikum dan masukkan ke bagian berikut.

HASIL TAMPILAN APLIKASI

A. Halaman Home

Halaman Home menampilkan kolom pencarian dengan petunjuk Cari game di bagian atas, diikuti daftar kartu permainan. Setiap kartu memuat thumbnail, judul, genre, dan ikon titik tiga. Pada gambar yang dilampirkan, daftar beserta gambar game telah tampil dan tidak terdapat pesan error yang terlihat.



Gambar 1. Tampilan Home dengan kolom pencarian dan daftar kartu game.

B. Halaman Detail

Halaman Detail menampilkan gambar utama sebuah game, panel Minimum System Requirements, galeri screenshot, dan cuplikan deskripsi. Panel kebutuhan sistem memuat OS, Processor, Memory, Graphics, serta Storage. Dokumentasi ini memperlihatkan penyajian informasi yang lebih lengkap dibandingkan kartu pada halaman Home.



Gambar 2. Tampilan Detail dengan gambar game, kebutuhan sistem, dan deskripsi.

KESIMPULAN

a. FutureBuilder menghubungkan proses pengambilan data asinkron dengan antarmuka, termasuk penanganan kondisi loading, error, dan data kosong.

b. ListView.builder dan Card menyediakan susunan daftar game yang konsisten. Pada contoh modul, tampilan dibatasi pada 25 data pertama dari respons API.

c. Navigator.pushNamed() meneruskan ID game dari Home ke Detail, sedangkan Navigator.pop() digunakan untuk kembali. Peta routes pada MaterialApp mengatur alur perpindahan halaman secara terpusat.

d. Model Game menggunakan factory fromJson() manual. Model DetailGame beserta model turunannya menggunakan json_serializable dan build_runner untuk menghasilkan fungsi konversi JSON.

e. Pembagian folder model, view, dan viewmodel memisahkan data, tampilan, serta fungsi jaringan, sehingga susunan kode lebih mudah dibaca dan dipelihara.

PENUTUP

Pembahasan Modul 10 memperlihatkan hubungan antara pembuatan GUI, navigasi bernama, dan integrasi API pada aplikasi game_app. Halaman Home menyajikan daftar permainan, sedangkan Detail menggunakan ID game untuk meminta informasi tambahan. Penggunaan model data membantu antarmuka bekerja dengan objek Dart, bukan dengan struktur JSON mentah.

Laporan ini memuat tujuan, langkah kerja, penjelasan kode utama, dokumentasi tampilan Home dan Detail, serta kesimpulan dari materi praktikum. Integrasi antarmuka dan model data menjadi dasar untuk memahami pengembangan aplikasi yang menggunakan informasi dari layanan eksternal. Dokumentasi kedua halaman melengkapi pembahasan proses penyajian daftar dan rincian game pada aplikasi.